

Universidade do Estado do Rio de Janeiro

Jully Moura

Atividade 7

Rio de Janeiro
2025

```
[7]: #Instalando e importando bibliotecas
!pip install uproot
import uproot
!pip install vector
import vector
!pip install hist
import hist
!pip install coffea
import coffea
import numpy as np
import awkward as ak
import matplotlib.pyplot as plt
```

```
[8]: #Localizando o arquivo a ser utilizado e abrindo para ver as componentes
path = '/opendata/eos/opendata/cms/mc/RunIISummer20UL16NanoAODv9/
↳ZZ_TuneCP5_13TeV-pythia8/NANOADSIM/106X_mcRun2_asymptotic_v17-v1/130000 '
file = uproot.open(path + "/76A312CE-F9C3-5F4D-9093-F26D7C50BF61.root")

print('Arquivo:', file.keys())
tree = file['Events;1']

tree.keys()
```

```
[10]: #Extraindo arrays dos dados de taus
tau_pt = tree['Tau_pt'].array()
tau_eta = tree['Tau_eta'].array()
tau_phi = tree['Tau_phi'].array()
tau_mass = tree['Tau_mass'].array()

#Filtrando eventos que possuem pelo menos 2 taus
valid_tau_pt = ak.num(tau_pt) >= 2

# Aplicando cortes no primeiro tau
mask_tau_pt = tau_pt[valid_tau_pt][:, 0] > 20
mask_tau_eta = np.abs(tau_eta[valid_tau_pt][:, 0]) < 2.5

# Aplicando cortes no segundo tau
mask_tau_pt2 = tau_pt[valid_tau_pt][:, 1] > 12
mask_tau_eta2 = np.abs(tau_eta[valid_tau_pt][:, 1]) < 2.5
mask_tau_combined = mask_tau_pt & mask_tau_eta & mask_tau_pt2 & mask_tau_eta2

#Combinando as máscaras para selecionar eventos válidos com taus que atendem aos
↳critérios
tau_pt_sel = tau_pt[valid_tau_pt][mask_tau_combined]
tau_eta_sel = tau_eta[valid_tau_pt][mask_tau_combined]
tau_phi_sel = tau_phi[valid_tau_pt][mask_tau_combined]
tau_mass_sel = tau_mass[valid_tau_pt][mask_tau_combined]
```

```
[11]: #Extraindo arrays dos dados de taus
muon_pt = tree['Muon_pt'].array()
muon_eta = tree['Muon_eta'].array()
muon_phi = tree['Muon_phi'].array()
muon_mass = tree['Muon_mass'].array()

#Filtrando eventos que possuem pelo menos 2 muons
valid_muon_pt = ak.num(muon_pt) >= 2

# Aplicando cortes no primeiro muon
mask_muon = (muon_pt[valid_muon_pt][:, 0] > 20) & (np.
    ↳abs(muon_eta[valid_muon_pt][:, 0]) < 2.5)
mask_muon2 = (muon_pt[valid_muon_pt][:, 1] > 12) & (np.
    ↳abs(muon_eta[valid_muon_pt][:, 1]) < 2.5)
mask_muon_combined = mask_muon & mask_muon2

#Combinando as máscaras para selecionar eventos válidos com muons que atendem
    ↳aos critérios
muon_pt_sel = muon_pt[valid_muon_pt][mask_muon_combined]
muon_eta_sel = muon_eta[valid_muon_pt][mask_muon_combined]
muon_phi_sel = muon_phi[valid_muon_pt][mask_muon_combined]
muon_mass_sel = muon_mass[valid_muon_pt][mask_muon_combined]
```

```
[12]: #Extrai arrays dos dados de eletrons
electron_pt = tree['Electron_pt'].array()
electron_eta = tree['Electron_eta'].array()
electron_phi = tree['Electron_phi'].array()
electron_mass = tree['Electron_mass'].array()

#Filtrando eventos que possuem pelo menos 2 eletrons
valid_electron_pt = ak.num(electron_pt) >= 2

# Aplicando cortes no primeiro eletron
mask_electron_pt = electron_pt[valid_electron_pt][:, 0] > 20
mask_electron_eta = np.abs(electron_eta[valid_electron_pt][:, 0]) < 2.5

# Aplicando cortes no segundo eletron
mask_electron_pt2 = electron_pt[valid_electron_pt][:, 1] > 12
mask_electron_eta2 = np.abs(electron_eta[valid_electron_pt][:, 1]) < 2.5
mask_electron_combined = mask_electron_pt & mask_electron_eta &
    ↳mask_electron_pt2 & mask_electron_eta2

# Combinando as máscaras para selecionar eventos válidos com eletrons que
    ↳atendem aos critérios
electron_pt_sel = electron_pt[valid_electron_pt][mask_electron_combined]
electron_eta_sel = electron_eta[valid_electron_pt][mask_electron_combined]
```

```
electron_phi_sel = electron_phi[valid_electron_pt][mask_electron_combined]
electron_mass_sel = electron_mass[valid_electron_pt][mask_electron_combined]
```

```
[13]: #Extrai arrays dos dados de jatos
jet_pt = tree['Jet_pt'].array()
jet_eta = tree['Jet_eta'].array()
jet_phi = tree['Jet_phi'].array()
jet_mass = tree['Jet_mass'].array()

#Filtrando eventos que possuem pelo menos 2 jatos
valid_jet_pt = ak.num(jet_pt) >= 2

# Aplicando cortes no primeiro jato
mask_jet_pt = jet_pt[valid_jet_pt][:, 0] > 20
mask_jet_eta = np.abs(jet_eta[valid_jet_pt][:, 0]) < 2.5

# Aplicando cortes no segundo jato
mask_jet_pt2 = jet_pt[valid_jet_pt][:, 1] > 12
mask_jet_eta2 = np.abs(jet_eta[valid_jet_pt][:, 1]) < 2.5
mask_jet_combined = mask_jet_pt & mask_jet_eta & mask_jet_pt2 & mask_jet_eta2

# Combinando as máscaras para selecionar eventos válidos com jatos que atendem
↪ aos critérios
jet_pt_sel = jet_pt[valid_jet_pt][mask_jet_combined]
jet_eta_sel = jet_eta[valid_jet_pt][mask_jet_combined]
jet_phi_sel = jet_phi[valid_jet_pt][mask_jet_combined]
jet_mass_sel = jet_mass[valid_jet_pt][mask_jet_combined]
```

```
[14]: #Removendo listas vazias
tau_pt_clean = tau_pt_sel[ak.num(tau_pt_sel) > 0]
tau_eta_clean = tau_eta_sel[ak.num(tau_eta_sel) > 0]
tau_phi_clean = tau_phi_sel[ak.num(tau_phi_sel) > 0]
tau_mass_clean = tau_mass_sel[ak.num(tau_mass_sel) > 0]
```

```
[15]: #Removendo listas vazias
muon_pt_clean = muon_pt_sel[ak.num(muon_pt_sel) > 0]
muon_eta_clean = muon_eta_sel[ak.num(muon_eta_sel) > 0]
muon_phi_clean = muon_phi_sel[ak.num(muon_phi_sel) > 0]
muon_mass_clean = muon_mass_sel[ak.num(muon_mass_sel) > 0]
```

```
[16]: #Removendo listas vazias
electron_pt_clean = electron_pt_sel[ak.num(electron_pt_sel) > 0]
electron_eta_clean = electron_eta_sel[ak.num(electron_eta_sel) > 0]
electron_phi_clean = electron_phi_sel[ak.num(electron_phi_sel) > 0]
electron_mass_clean = electron_mass_sel[ak.num(electron_mass_sel) > 0]
```

```
[17]: #Removendo listas vazias
jet_pt_clean = jet_pt_sel[ak.num(jet_pt_sel) > 0]
jet_eta_clean = jet_eta_sel[ak.num(jet_eta_sel) > 0]
jet_phi_clean = jet_phi_sel[ak.num(jet_phi_sel) > 0]
jet_mass_clean = jet_mass_sel[ak.num(jet_mass_sel) > 0]

[18]: # Transformando array em 1D para ser possível rodar o plot
tau_pt_flat = ak.flatten(tau_pt_clean)
tau_eta_flat = ak.flatten(tau_eta_clean)
tau_phi_flat = ak.flatten(tau_phi_clean)
tau_mass_flat = ak.flatten(tau_mass_clean)

[19]: # Transformando array em 1D para ser possível rodar o plot
muon_pt_flat = ak.flatten(muon_pt_clean)
muon_eta_flat = ak.flatten(muon_eta_clean)
muon_phi_flat = ak.flatten(muon_phi_clean)
muon_mass_flat = ak.flatten(muon_mass_clean)

[20]: # Transformando array em 1D para ser possível rodar o plot
electron_pt_flat = ak.flatten(electron_pt_clean)
electron_eta_flat = ak.flatten(electron_eta_clean)
electron_phi_flat = ak.flatten(electron_phi_clean)
electron_mass_flat = ak.flatten(electron_mass_clean)

[21]: # Transformando array em 1D para ser possível rodar o plot
jet_pt_flat = ak.flatten(jet_pt_clean)
jet_eta_flat = ak.flatten(jet_eta_clean)
jet_phi_flat = ak.flatten(jet_phi_clean)
jet_mass_flat = ak.flatten(jet_mass_clean)

[22]: #Funcao para criar quadrivetores
def create_4vecs(pt, eta, phi, mass, indices):
    """Função para criar vetores 4-momento e calcular a massa combinada."""
    return vector.zip({
        "pt": pt[:, indices],
        "eta": eta[:, indices],
        "phi": phi[:, indices],
        "mass": mass[:, indices]
    })

def calculate_mass(particle1_4vecs, particle2_4vecs):
    """Função para somar dois vetores de 4-momento e calcular a massa combinada.
    ↪ """
    combined = particle1_4vecs + particle2_4vecs
    return combined.mass
```

```
[23]: tau1_4vecs = create_4vecs(tau_pt_clean, tau_eta_clean, tau_phi_clean,
    ↪tau_mass_clean, 0)
tau2_4vecs = create_4vecs(tau_pt_clean, tau_eta_clean, tau_phi_clean,
    ↪tau_mass_clean, 1)
mass_ditau = calculate_mass(tau1_4vecs, tau2_4vecs)

[24]: muon1_4vecs = create_4vecs(muon_pt_clean, muon_eta_clean, muon_phi_clean,
    ↪muon_mass_clean, 0)
muon2_4vecs = create_4vecs(muon_pt_clean, muon_eta_clean, muon_phi_clean,
    ↪muon_mass_clean, 1)
mass_dimuon = calculate_mass(muon1_4vecs, muon2_4vecs)

[25]: electron1_4vecs = create_4vecs(electron_pt_clean, electron_eta_clean,
    ↪electron_phi_clean, electron_mass_clean, 0)
electron2_4vecs = create_4vecs(electron_pt_clean, electron_eta_clean,
    ↪electron_phi_clean, electron_mass_clean, 1)
mass_dielelectron = calculate_mass(electron1_4vecs, electron2_4vecs)

[26]: jet1_4vecs = create_4vecs(jet_pt_clean, jet_eta_clean, jet_phi_clean,
    ↪jet_mass_clean, 0)
jet2_4vecs = create_4vecs(jet_pt_clean, jet_eta_clean, jet_phi_clean,
    ↪jet_mass_clean, 1)
mass_dijet = calculate_mass(jet1_4vecs, jet2_4vecs)

[27]: # Definindo variáveis antes dos cortes
tau_pt_before_cut = tree['Tau_pt'].array()
tau_pt_before_cut_clean = tau_pt_before_cut[ak.num(tau_pt_before_cut) > 0]
tau_pt_before = ak.flatten(tau_pt_before_cut_clean)

tau_eta_before_cut = tree['Tau_eta'].array()
tau_eta_before_cut_clean = tau_eta_before_cut[ak.num(tau_eta_before_cut) > 0]
tau_eta_before = ak.flatten(tau_eta_before_cut_clean)

[28]: # Definindo variáveis antes dos cortes
muon_pt_before_cut = tree['Muon_pt'].array()
muon_pt_before_cut_clean = muon_pt_before_cut[ak.num(muon_pt_before_cut) > 0]
muon_pt_before = ak.flatten(muon_pt_before_cut_clean)

muon_eta_before_cut = tree['Muon_eta'].array()
muon_eta_before_cut_clean = muon_eta_before_cut[ak.num(muon_eta_before_cut) > 0]
muon_eta_before = ak.flatten(muon_eta_before_cut_clean)

[29]: # Definindo variáveis antes dos cortes
electron_pt_before_cut = tree['Electron_pt'].array()
electron_pt_before_cut_clean = electron_pt_before_cut[ak.
    ↪num(electron_pt_before_cut) > 0]
electron_pt_before = ak.flatten(electron_pt_before_cut_clean)
```

```

electron_eta_before_cut = tree['Electron_eta'].array()
electron_eta_before_cut_clean = electron_eta_before_cut[ak.
    ↳num(electron_eta_before_cut) > 0]
electron_eta_before = ak.flatten(electron_eta_before_cut_clean)

```

```

[30]: # Definindo variáveis antes dos cortes
jet_pt_before_cut = tree['Jet_pt'].array()
jet_pt_before_cut_clean = jet_pt_before_cut[ak.num(jet_pt_before_cut) > 0]
jet_pt_before = ak.flatten(jet_pt_before_cut_clean)

jet_eta_before_cut = tree['Jet_eta'].array()
jet_eta_before_cut_clean = jet_eta_before_cut[ak.num(jet_eta_before_cut) > 0]
jet_eta_before = ak.flatten(jet_eta_before_cut_clean)

```

```

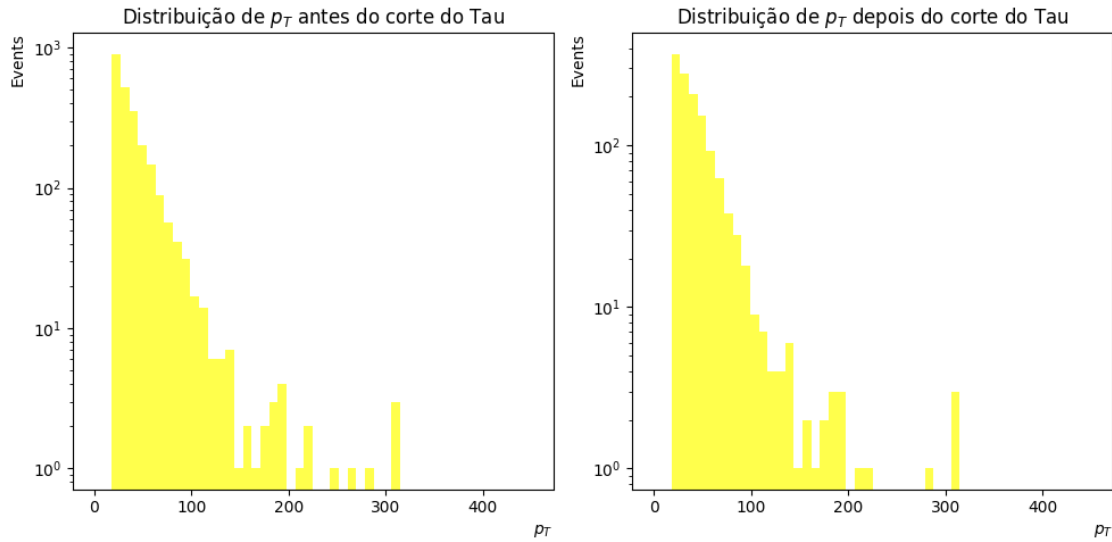
[31]: #Plotando os gráficos do pt de tau de antes e depois do corte
plt.figure(figsize=(10, 5))

plt.subplot(1, 2, 1)
plt.hist(tau_pt_before, bins=50, color='#FFFF00', alpha=0.7, range=(0, 450))
plt.title("Distribuição de $p_T$ antes do corte do Tau")
plt.xlabel(r"$p_T$", loc='right')
plt.ylabel("Events", loc='top')
plt.yscale('log')

plt.subplot(1, 2, 2)
plt.hist(tau_pt_flat, bins=50, color='#FFFF00', alpha=0.7, range=(0, 450))
plt.title("Distribuição de $p_T$ depois do corte do Tau")
plt.xlabel(r"$p_T$", loc='right')
plt.ylabel("Events", loc='top')
plt.yscale('log')

plt.tight_layout()
plt.show()

```

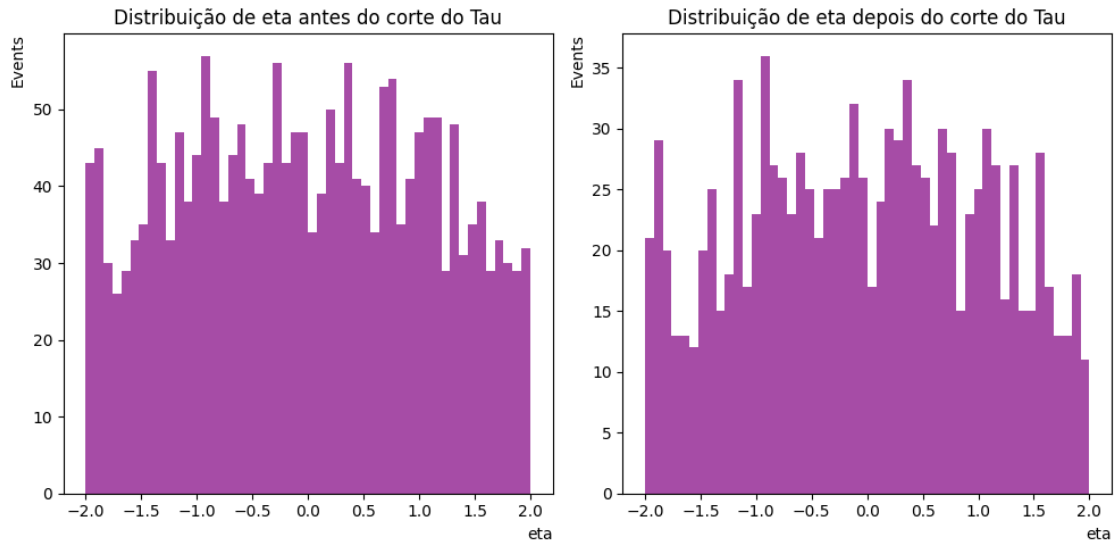


```
[32]: #Plotando os gráficos do eta de tau de antes e depois do corte
plt.figure(figsize=(10, 5))

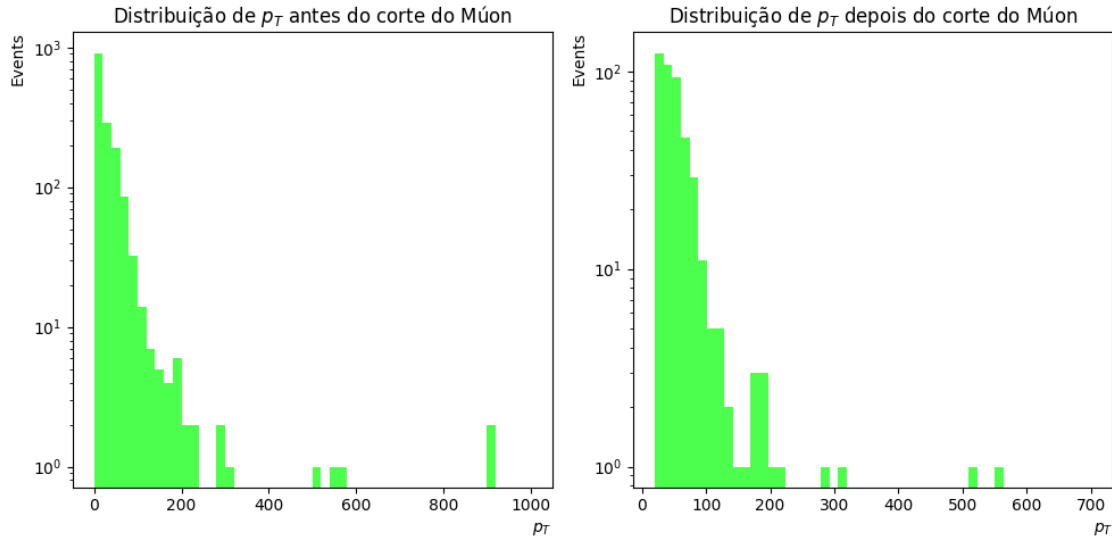
plt.subplot(1, 2, 1)
plt.hist(tau_eta_before, bins=50, color='#800080', alpha=0.7, range=(-2, 2))
plt.title("Distribuição de eta antes do corte do Tau")
plt.xlabel(r"eta", loc='right')
plt.ylabel("Events", loc='top')

plt.subplot(1, 2, 2)
plt.hist(tau_eta_flat, bins=50, color='#800080', alpha=0.7, range=(-2, 2))
plt.title("Distribuição de eta depois do corte do Tau")
plt.xlabel(r"eta", loc='right')
plt.ylabel("Events", loc='top')

plt.tight_layout()
plt.show()
```

```
[33]: #Plotando os gráficos do pt de muon de antes e depois do corte
plt.figure(figsize=(10, 5))
plt.subplot(1, 2, 1)
plt.hist(ak.flatten(tree['Muon_pt'].array()[ak.num(tree['Muon_pt'].array()) > 0]), bins=50, color='#00FF00', alpha=0.7, range=(0, 1000))
plt.yscale('log')
plt.title(r'Distribuição de $p_T$ antes do corte do Múon')
plt.xlabel(r"$p_{T}$", loc='right')
plt.ylabel("Events", loc='top')
plt.subplot(1, 2, 2)
plt.hist(ak.flatten(muon_pt_clean), bins=50, color='#00FF00', alpha=0.7, range=(20, 700))
plt.yscale('log')
plt.title(r'Distribuição de $p_T$ depois do corte do Múon')
plt.xlabel(r"$p_{T}$", loc='right')
plt.ylabel("Events", loc='top')
plt.tight_layout()
plt.show()
```

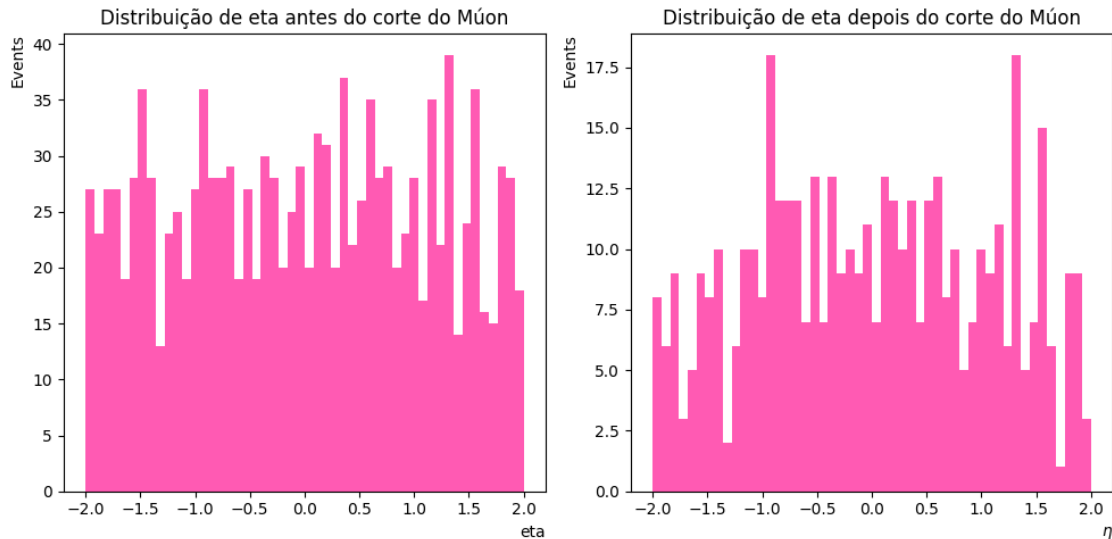


```
[34]: #Plotando os gráficos do eta de muon de antes e depois do corte
plt.figure(figsize=(10, 5))

plt.subplot(1, 2, 1)
plt.hist(muon_eta_before, bins=50, color='#FF1493', alpha=0.7, range=(-2, 2))
plt.title("Distribuição de eta antes do corte do Múon")
plt.xlabel(r"eta", loc='right')
plt.ylabel("Events", loc='top')

plt.subplot(1, 2, 2)
plt.hist(muon_eta_flat, bins=50, color='#FF1493', alpha=0.7, range=(-2, 2))
plt.title("Distribuição de eta depois do corte do Múon")
plt.xlabel(r"$\eta$", loc='right')
plt.ylabel("Events", loc='top')

plt.tight_layout()
plt.show()
```

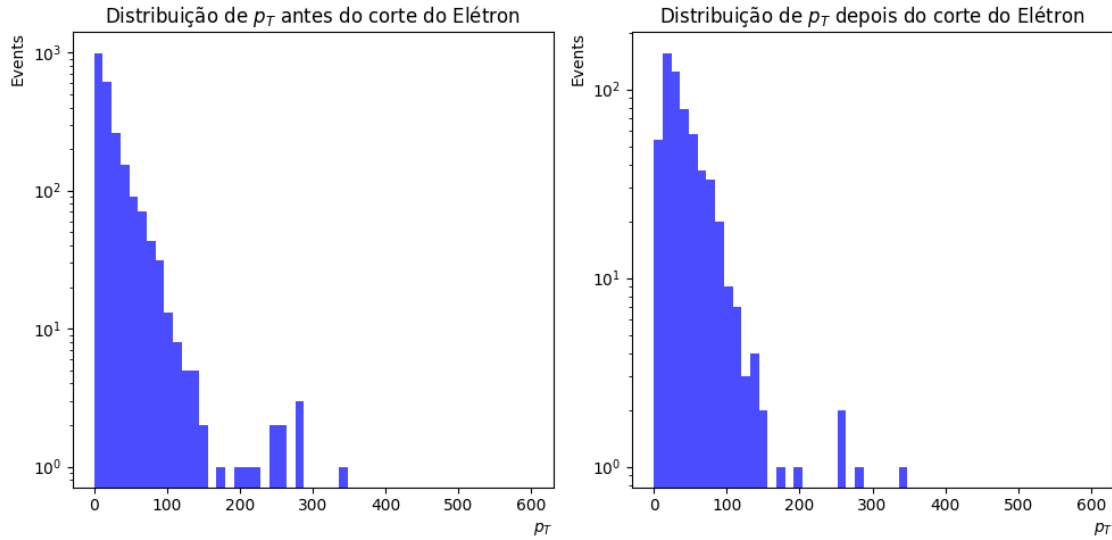


```
[35]: #Plotando os gráficos do pt do eletron de antes e depois do corte
plt.figure(figsize=(10, 5))

plt.subplot(1, 2, 1)
plt.hist(electron_pt_before, bins=50, color='#0000FF', alpha=0.7, range=(0, 600))
plt.title(r"Distribuição de $p_T$ antes do corte do Elétron")
plt.xlabel(r"$p_T$", loc='right')
plt.ylabel("Events", loc='top')
plt.yscale('log')

plt.subplot(1, 2, 2)
plt.hist(electron_pt_flat, bins=50, color='#0000FF', alpha=0.7, range=(0, 600))
plt.title(r"Distribuição de $p_T$ depois do corte do Elétron")
plt.xlabel(r"$p_T$", loc='right')
plt.ylabel("Events", loc='top')
plt.yscale('log')

plt.tight_layout()
plt.show()
```

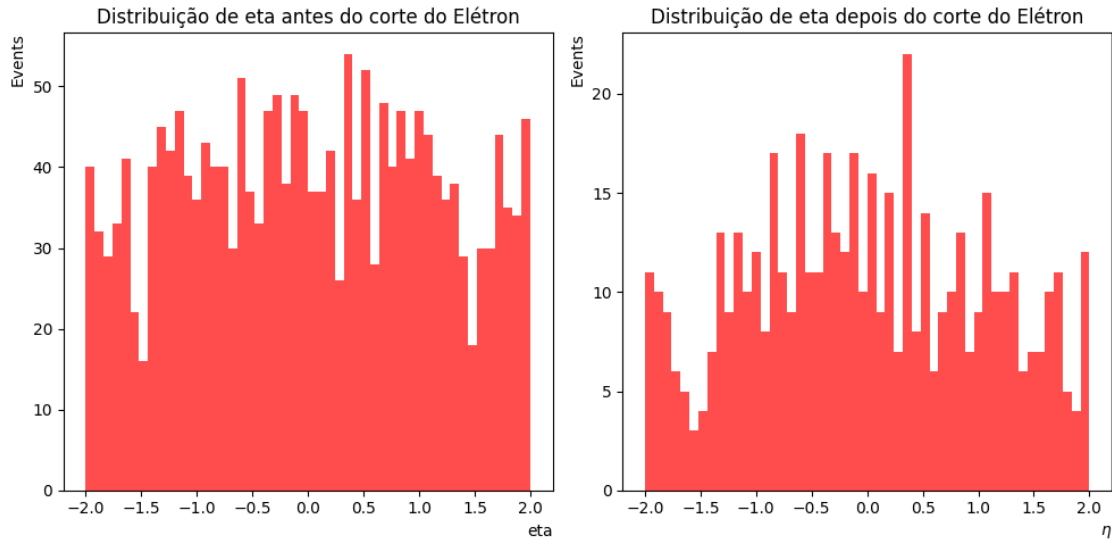


```
[36]: #Plotando os gráficos do eta do eletron de antes e depois do corte
plt.figure(figsize=(10, 5))

plt.subplot(1, 2, 1)
plt.hist(electron_eta_before, bins=50, color='#FF0000', alpha=0.7, range=(-2, 2))
plt.title("Distribuição de eta antes do corte do Elétron")
plt.xlabel(r"eta", loc='right')
plt.ylabel("Events", loc='top')

plt.subplot(1, 2, 2)
plt.hist(electron_eta_flat, bins=50, color='#FF0000', alpha=0.7, range=(-2, 2))
plt.title("Distribuição de eta depois do corte do Elétron")
plt.xlabel(r"$\eta$", loc='right')
plt.ylabel("Events", loc='top')

plt.tight_layout()
plt.show()
```

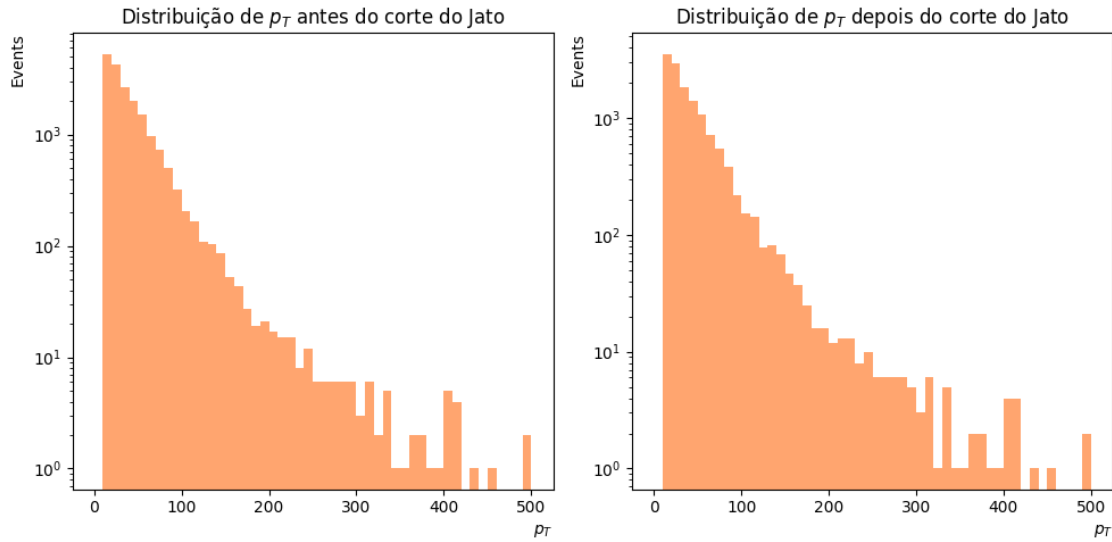


```
[37]: #Plotando os gráficos do pt do jato de antes e depois do corte
plt.figure(figsize=(10, 5))

plt.subplot(1, 2, 1)
plt.hist(jet_pt_before, bins=50, color='#FF7F32', alpha=0.7, range=(0, 500))
plt.title("Distribuição de $p_T$ antes do corte do Jato")
plt.xlabel(r"$p_T$", loc='right')
plt.ylabel("Events", loc='top')
plt.yscale('log')

plt.subplot(1, 2, 2)
plt.hist(jet_pt_flat, bins=50, color='#FF7F32', alpha=0.7, range=(0, 500))
plt.title("Distribuição de $p_T$ depois do corte do Jato")
plt.xlabel(r"$p_T$", loc='right')
plt.ylabel("Events", loc='top')
plt.yscale('log')

plt.tight_layout()
plt.show()
```

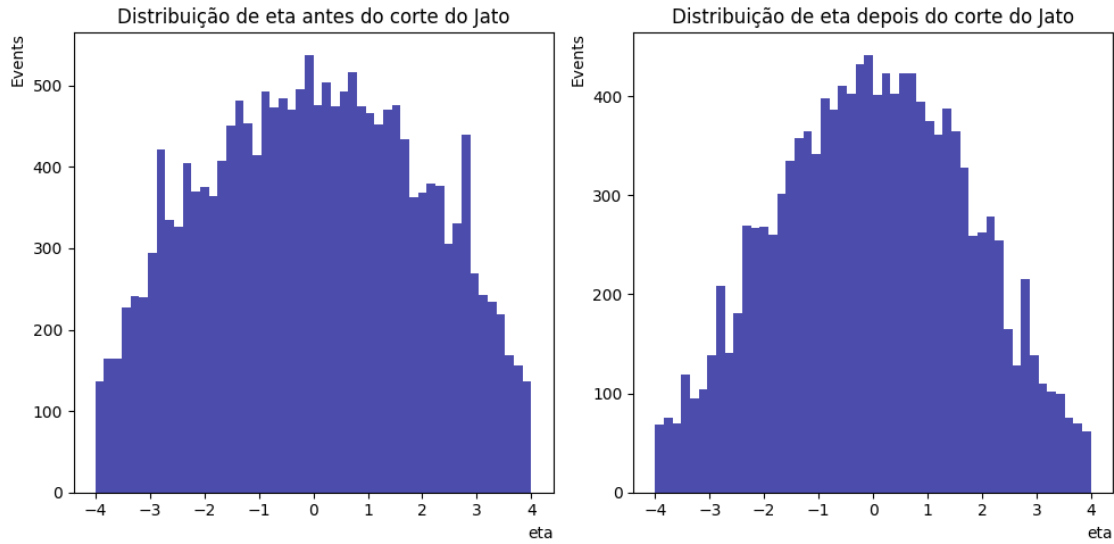


```
[38]: #Plotando os gráficos do eta do jato de antes e depois do corte
plt.figure(figsize=(10, 5))

plt.subplot(1, 2, 1)
plt.hist(jet_eta_before, bins=50, color='#00008B', alpha=0.7, range=(-4, 4))
plt.title("Distribuição de eta antes do corte do Jato")
plt.xlabel(r"eta", loc='right')
plt.ylabel("Events", loc='top')

plt.subplot(1, 2, 2)
plt.hist(jet_eta_flat, bins=50, color='#00008B', alpha=0.7, range=(-4, 4))
plt.title("Distribuição de eta depois do corte do Jato")
plt.xlabel(r"eta", loc='right')
plt.ylabel("Events", loc='top')

plt.tight_layout()
plt.show()
```

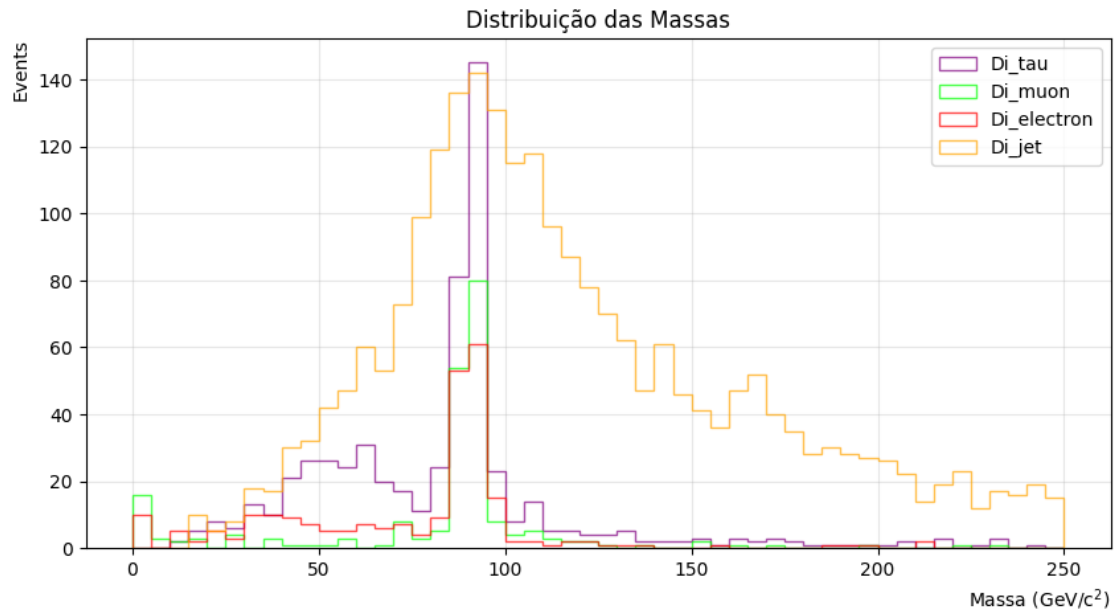


[39]: *# Plotando o gráfico da distribuição das massas*

```
plt.figure(figsize=(10, 5))

# Plot de cada variável
plt.hist(mass_ditau, bins=50, color='#800080', alpha=0.7, label='Di_tau',
         histtype='step', range = (0,250))
plt.hist(mass_dimuon, bins=50, color='#00FF00', alpha=0.7, label='Di_muon',
         histtype='step', range = (0,250))
plt.hist(mass_dielelectron, bins=50, color='#FF0000', alpha=0.7,
         label='Di_electron', histtype='step', range = (0,250))
plt.hist(mass_dijet, bins=50, color='#FFA500', alpha=0.7, label='Di_jet',
         histtype='step', range = (0,250))

# Configurações do gráfico
plt.title("Distribuição das Massas")
plt.xlabel(r"Massa (GeV/c$^2$)", loc='right')
plt.ylabel("Events", loc='top')
plt.legend(loc='best')
plt.grid(alpha=0.3)
plt.show()
```



Resultados

Verificando os plots acima, pode-se perceber que o número de eventos foi afetado, pois como foi feito um corte, alguns eventos não atendiam aos critérios e foram descartados pelo código.